

API preparation

Table of Contents

- 1.1 API Authentication
- 1.2 Status Codes
- 1.3 Human Handoff Webhook Event

API Authentication

MaiAgent uses API keys for authentication.

Authentication Method

All API requests must include an API Key in the HTTP Header, in the following format:

```
const headers = {  
  "Authorization": "Api-Key YOUR_API_KEY_HERE",  
  "Content-Type": "application/json"  
}
```

How to Obtain an API Key?

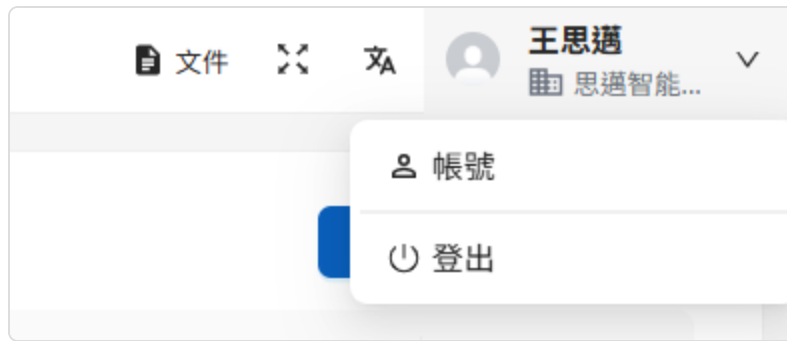
Please log in to the MaiAgent system backend and follow these steps:

Click the "Username" dropdown menu in the upper right corner.

Click "Account" to go to the profile page.

The screenshot displays the MaiAgent system backend interface. The top navigation bar includes the MaiAgent logo, a menu icon, and the text "AI 功能 / AI 助理". On the right side of the top bar, there is a user profile dropdown menu with the name "王思涵" and a sub-item "思邁智能...". The left sidebar contains a navigation menu with items: "思邁智能股份有限...", "AI 功能", "AI 助理", "知識庫", "工具", and "爬蟲". The main content area shows a table of AI assistants. The table has columns for ID, Name, Large Language Model, Response Mode, Associated Knowledge Base, Last Update Time, and Actions. There are three rows of data in the table. Above the table, there are filters for "請輸入 AI 助理名稱或 ID", "全部大語言模型", "全部回答模式", and "重置篩選". A button "建立 AI 助理" is located to the right of the filters. At the bottom right of the page, there is a status bar showing "共 3 條資料" and "10 條/頁".

ID	名稱	大語言模型	回答模式	關聯知識庫	最後更新時間	操作
1af8b729-cbca-4c07-b685-...	LLM 測試用	Qwen3 32B(groq)	一般(預設)	LLM 測試用的知識庫	2025-09-09 23:27:51	[Icon] [Icon]
da10bf5e-b8c8-4cac-85cf-...	全螢幕嵌入示範	Claude 4 Sonnet	Agent	露營資料庫	2025-08-27 18:01:48	[Icon] [Icon]
68aa8a32-798a-473c-8cc3-...	台股大盤分析	Claude 4 Sonnet	Agent	台股大盤分析的知識庫	2025-08-12 10:40:59	[Icon] [Icon]



Switch the page to the API Key page.



You can then copy and view the "API Key".



Security Reminder: Please keep your API key secure and do not share it in public places. Key Format Description: The API key is a combination of alphanumeric characters.

Error Handling

When authentication fails, the API will return a 401 Unauthorized HTTP error code:

Invalid or Missing API Key

```
{
  "detail": "Authentication credentials were not provided."
}
```

Fix suggestion: Check if the request header contains the correct Authorization field. The format should be: `Authorization: Api-Key YOUR_API_KEY`

Incorrect API Key Format

```
{
  "detail": "Invalid API key format."
}
```

Fix suggestion: Confirm that the API Key format is correct. Check if the correct prefix `Api-Key` is used instead of `Bearer`, and ensure the API Key itself does not have extra spaces or characters.

Status Codes

When sending an HTTP request via the API, you can use the following status codes to check if the API request was successful:

Status Code Descriptions

Success Codes

Status Code	Description
200	Request successful
201	Resource created successfully

Success Response Example

This is a content demonstration. The response format may vary for different endpoints. Please refer to the documentation for the specific endpoint.

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // Result content returned
  ]
}
```

Error Codes

Status Code	Description
400	Bad Request
401	Authentication failed
403	Permission denied

Status Code	Description
404	Resource not found
500	Server error

Error Response Example

400 - Bad Request: Required field missing

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

Human Handoff Webhook Event

When a human agent is assigned to a conversation in a customer service inbox, MaiAgent sends a `handoff.created` event to your configured endpoint via HTTPS POST. This allows dispatch, alerting, or CRM systems to take over automatically without polling. When the conversation returns to AI, the same endpoint receives `handoff.returned_to_ai`, notifying the system handling the conversation that the human handoff has ended.

How to Enable

The customer's administrator or integration specialist can configure this in the admin console without engineering assistance:

Go to Admin console → Settings → Inboxes → select an inbox → "Human Handoff" tab → "Notifications" section.

Turn on "Webhook Notifications (System Integration)" and enter the receiving endpoint URL (only URLs beginning with `https://` are accepted).

(Optional but recommended) Enter a signing secret to enable source verification (see "Signature Verification" below).

Webhook, Notification Center, and Email are three independent settings and do not affect one another. They are disabled by default and enabled per inbox. This channel is separate from the inbox's existing "Message Webhook" for sending and receiving messages.

Event Types

event	When it is sent	Notes
<code>handoff.created</code>	A conversation is assigned to a human agent (including manual assignment, self-assignment, transfer, and automatic assignment)	<code>assigneeId</code> identifies the assigned agent; another event is sent when the conversation is transferred to a different agent
<code>handoff.returned_to_ai</code>	An agent clicks "Return to AI Agent" in the admin console or returns the conversation to AI through the <code>return-to-ai</code> API	<code>assigneeId</code> is <code>null</code> because the conversation no longer has an assigned agent

Both event types use the same payload fields, signature, and retry behavior. The receiver must **route by the `event` field**. More event types may be added in the future; return a 2xx response and ignore any unrecognized `event` instead of treating it as an error.

`handoff.returned_to_ai` means that an **agent explicitly returned the conversation to AI**. It does not mean that every human handoff flow has ended. This event is not sent when an agent marks the conversation as resolved or when the conversation returns to the queue because the agent loses access to the inbox. If you need a complete resolution signal, query `status` through the Conversations API or monitor state changes in `conversation.progress_status` from the Message Webhook.

Event Format

POST
Content-Type: application/json

```
{
  "event": "handoff.created",
  "conversationId": "6c2b7c1e- ... ",
  "inboxId": "0f9a3d42- ... ",
  "organizationId": "b1d20c77- ... ",
  "assigneeId": "3e8f5a90- ... ",
  "contactId": "a4c1f6b3- ... ",
  "conversationUrl": "https://admin.maiaagent.ai/ ... "
}
```

Field	Type	Description
event	string	<code>handoff.created</code> or <code>handoff.returned_to_ai</code> (see "Event Types" above)
conversationId	string (uuid)	Conversation identifier
inboxId	string (uuid)	Inbox identifier
organizationId	string (uuid)	Organization identifier
assigneeId	string (uuid)	null
contactId	string (uuid)	null
conversationUrl	string (uri)	Link to the conversation page in the admin console

The payload contains only identifiers and the admin console link, **not conversation text**. To retrieve conversation content, use the identifiers to query the Conversations and Messages APIs.

Signature Verification (Recommended)

After you configure a signing secret in the admin console, every request includes this header:

```
X-MaiAgent-Signature-256: sha256=
```

The value is the hexadecimal HMAC-SHA256 digest of the **raw request body bytes**, calculated using the secret. Always verify the exact raw bytes received. Do not parse and reserialize the body first, because differences in field order or whitespace will change the digest.

```
import hashlib
import hmac

def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
    hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

The secret is write-only: it is not shown after it is saved, and the admin console displays only "Configured." Saving with the field empty preserves the existing secret. Enter a new value to replace it, or use "Clear Secret" to remove it. Requests do not include the signature header when no secret is configured.

Minimal Receiver Example

The following runnable Python (Flask) receiver reads the raw bytes, verifies the signature, routes by `event`, and returns a 2xx response before performing time-consuming work. Replace `HANDOFF_WEBHOOK_SECRET` with the signing secret configured in the admin console.

```

import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]

def verify_signature(raw_body: bytes, header_value: str | None) → bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)

@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data() # Use the raw bytes; do not reserialize
request.json
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    key = (event["event"], event["conversationId"]) # Retries contain identical
data; use this pair for deduplication

    if event["event"] == "handoff.created":
        # Open a ticket/session in your system. A transfer sends another event
with a different assigneeId
        open_ticket(event["conversationId"], event["assigneeId"],
event["conversationUrl"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(event["conversationId"])
    # Unknown event: return 2xx and ignore it instead of treating it as an error

    return "", 204 # Return 2xx within 10 seconds; send time-consuming work to
the background

def open_ticket(conversation_id, assignee_id, url):
    ...

```

```
def close_ticket(conversation_id):  
    ...
```

```
HANDOFF_WEBHOOK_SECRET=your-secret flask --app receiver run --port 8080
```

Use the following command to simulate a request locally. The signature is calculated using the same secret.

```
BODY='{ "event": "handoff.created", "conversationId": "6c2b7c1e-0000-0000-0000-000000000001", "inboxId": "0f9a3d42-0000-0000-0000-000000000002", "organizationId": "b1d20c77-0000-0000-0000-000000000003", "assigneeId": "3e8f5a90-0000-0000-0000-000000000004", "contactId": "a4c1f6b3-0000-0000-0000-000000000005", "conversationUrl": "https://admin.maiagent.ai/conversations/index?conversationId=6c2b7c1e-0000-0000-0000-000000000001"}'  
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "your-secret" | awk '{print $NF}')
```

```
curl -X POST http://localhost:8080/webhooks/handoff  
  -H "Content-Type: application/json"  
  -H "X-MaiAgent-Signature-256: sha256=$SIG"  
  -d "$BODY"
```

Responses and Retries

The receiving endpoint must respond with a 2xx status code within 10 seconds. The response body can be empty.

On a timeout, connection failure, or non-2xx response, the system retries automatically up to 3 times at 60-second intervals. Retries contain exactly the same content as the original request, so the receiver must handle them idempotently (for example, deduplicate by `event` + `conversationId`).

A failed Webhook request does not affect the human handoff flow, Notification Center, or Email notifications.

External System Takeover After Human Handoff

If your human agents reply from their own customer service system rather than the MaiAgent admin console, combine the events on this page with the inbox's existing Message Webhook and the Messages API to synchronize the conversation in both directions during the handoff. All steps use existing capabilities and require no additional activation.

Integration Flow

Detect the start of a human handoff: After receiving `handoff.created` , open a ticket or session in your system using `conversationId` , and record `assigneeId` and `contactId` .

Receive customer messages and agent replies: In the admin console, add Message Webhooks for both `incoming` (customer messages) and `outgoing` (AI or agent replies) directions to the inbox. Each message's `conversation` object contains the current processing state. Process only messages whose `progress_status` is `human_serving` to exclude AI-mode traffic:

Field	Description
conversation.status	<code>open</code> <code>queued</code> <code>resolved</code>
conversation.progress_status	<code>ai_processing</code> <code>waiting_for_human</code> <code>human_serving</code> <code>resolved</code> <code>open</code>
conversation.auto_reply_enabled	Whether AI replies automatically; <code>false</code> after human handoff
conversation.assignee	Assigned agent <code>{ "id", "name" }</code> ; <code>null</code> when unassigned
sender.type	<code>contact</code> (customer) <code>user</code> (human agent) <code>chatbot</code> (AI)

Write agent replies back to the conversation: Call `POST /api/v1/messages/outgoing/` with an API key and include `conversation` and `content` . MaiAgent forwards the message to the customer's channel. The sender is the member who owns the API key, and that member must have access to the inbox (see the Conversations and Messages endpoint documentation).

End the human handoff: After the agent finishes, call `PATCH /api/v1/conversations/{id}/return-to-ai/` so AI can take over again. If the agent clicks "Return to AI Agent" in the MaiAgent admin console, you receive `handoff.returned_to_ai` ; use it to close the session in your system.

Request Examples

(1) Message Webhook received during a human handoff (`maiaagent_standard` format, abbreviated) —use `conversation.progress_status` to determine whether the conversation is in human mode and `sender.type` to identify the sender:

```
{
  "id": "9d2f7a10- ... ",
  "type": "incoming",
  "content": "I need to change my order address",
  "created_at": "2026-09-09T02:30:00+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "title": " ... ",
    "type": "individual",
    "status": "open",
    "progress_status": "human_serving",
    "auto_reply_enabled": false,
    "assignee": { "id": "3e8f5a90- ... ", "name": "Agent Amy" }
  },
  "inbox": { "id": "0f9a3d42- ... ", "name": "Website Support", "channel_type":
"web" },
  "contact": { "id": "a4c1f6b3- ... ", "name": "John Smith", "email": null,
"phone_number": null },
  "sender": { "id": "a4c1f6b3- ... ", "name": "John Smith", "type": "contact" },
  "attachments": [],
  "metadata": {}
}
```

(2) Write an agent reply back to the conversation—only `conversation` and `content` are required; other fields are optional. The message is sent to the customer's channel as the member who owns the API key:

```
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"
-H "Authorization: Api-Key YOUR_API_KEY"
-H "Content-Type: application/json"
-d '{
  "conversation": "6c2b7c1e- ... ",
  "content": "Hello, your address has been updated."
}'
```

(3) Return the conversation to AI after completing the request—no request body is required. A successful request returns 200 with the conversation object. AI then resumes automatic replies and `assignee` is cleared:

```
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/6c2b7c1e- ... /return-to-ai/"
-H "Authorization: Api-Key YOUR_API_KEY"
```

The following Python (`requests`) example combines all three steps:

```
import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": "Api-Key YOUR_API_KEY"}

def on_message_webhook(payload: dict) → None:
    conv = payload["conversation"]
    if conv["progress_status"] ≠ "human_serving":
        return # Do not send AI-mode traffic to the customer service system
    if payload["sender"]["type"] = "contact":
        push_to_agent_console(conv["id"], payload["content"]) # Customer message
        → your agent interface

def reply(conversation_id: str, text: str) → None:
    requests.post(f"{API}/messages/outgoing/", headers=HEADERS,
                  json={"conversation": conversation_id, "content": text},
                  timeout=10).raise_for_status()

def finish(conversation_id: str) → None:
    requests.patch(f"{API}/conversations/{conversation_id}/return-to-ai/",
                  headers=HEADERS, timeout=10).raise_for_status()
```

Notes

A reply written through the API is also delivered through the `outgoing` Message Webhook (`sender.type` is `user`). Deduplicate by message `id` to avoid displaying it twice in your system.

Another `handoff.created` event is sent when a conversation is transferred to a different agent, with `assigneeId` set to the new agent. Update the existing session by `conversationId` instead of opening a duplicate ticket.

The Message Webhook and human handoff Webhook have separate settings and signature methods. The former sends the configured secret unchanged in the `X-Webhook-Secret` header; the latter uses the HMAC-SHA256 signature described on this page.

